



Computer Science

02. Computer Architecture

2주차: 컴퓨터구조론

오늘의 핵심 질문

? 핵심 질문 (1주차와 동일)

"우리가 작성한 `hello.py` 파일은 저장장치에 잠들어 있다가 어떻게 실행되어 화면에 글자를 띄울까?"

.... 를 조금 더 자세하게 알아봅시다

목차 INDEX

01 CPU

- 2진수와 16진수
- 논리게이트 기초
- ALU
- 캐시 & 레지스터
- CPU 구조

02 컴퓨터구조론

- RAM과 SSD의 차이
- 메모리 주소
- 메모리 계층 구조
- 폰 노이만 구조와 하버드 구조

03 운영체제

- 시스템 계층 구조
- 커널
- 셸
- 사용자 모드와 커널 모드
- 시스템 콜

04 코드 실행 과정

- 컴파일러와 인터프리터
- Python이 내부에서 실행되는 과정
- 프로그래밍 언어의 급

01

Review

지난 내용 복습

각 부품의 역할

CPU

계산하고 명령하는 숙련된 작업자

실제로 생각하고 계산하는 두뇌입니다.
엄청나게 빠른 속도로 연산을 수행하지만,
책상이 없으면 일할 수 없습니다.

RAM

현재 일하는 넓은 작업용 책상

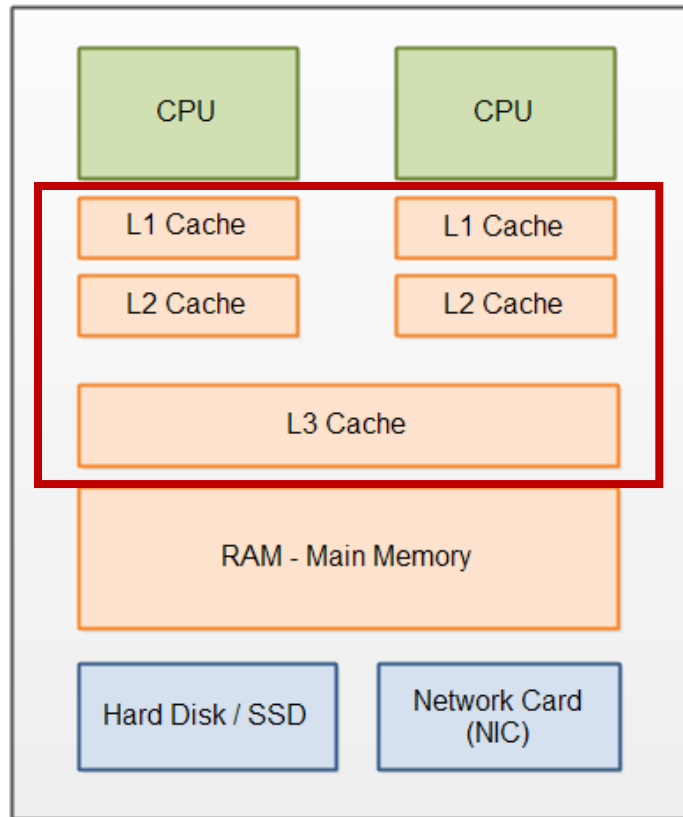
지금 당장 실행 중인 프로그램과 서류를
올려두는 책상입니다. 책상이 넓을수록
많은 프로그램을 동시에 띄울 수 있습니다.

SSD

자료를 보관하는 거대한 서랍장

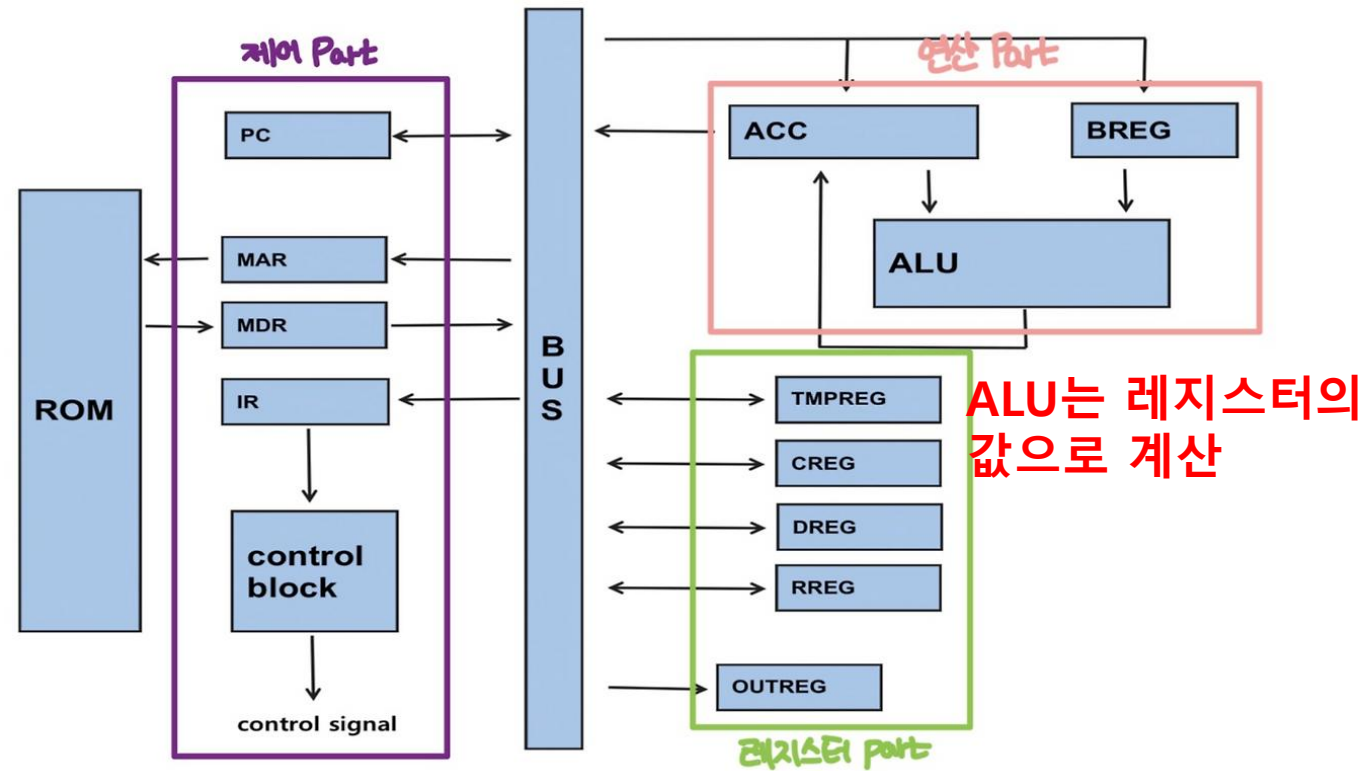
전원이 꺼져도 자료가 지워지지 않는 창고입니다.
평소 쓰지 않는 파이썬 파일, 게임, 사진 등이
안전하게 보관됩니다.

Cache & Register



L1, L2, L3
캐시 존재

- CPU 칩 안에 들어있는 고속 임시 저장소
- RAM에서 가져온 데이터 임시 저장
- 빠른 CPU의 속도를 맞추기 위해 필요

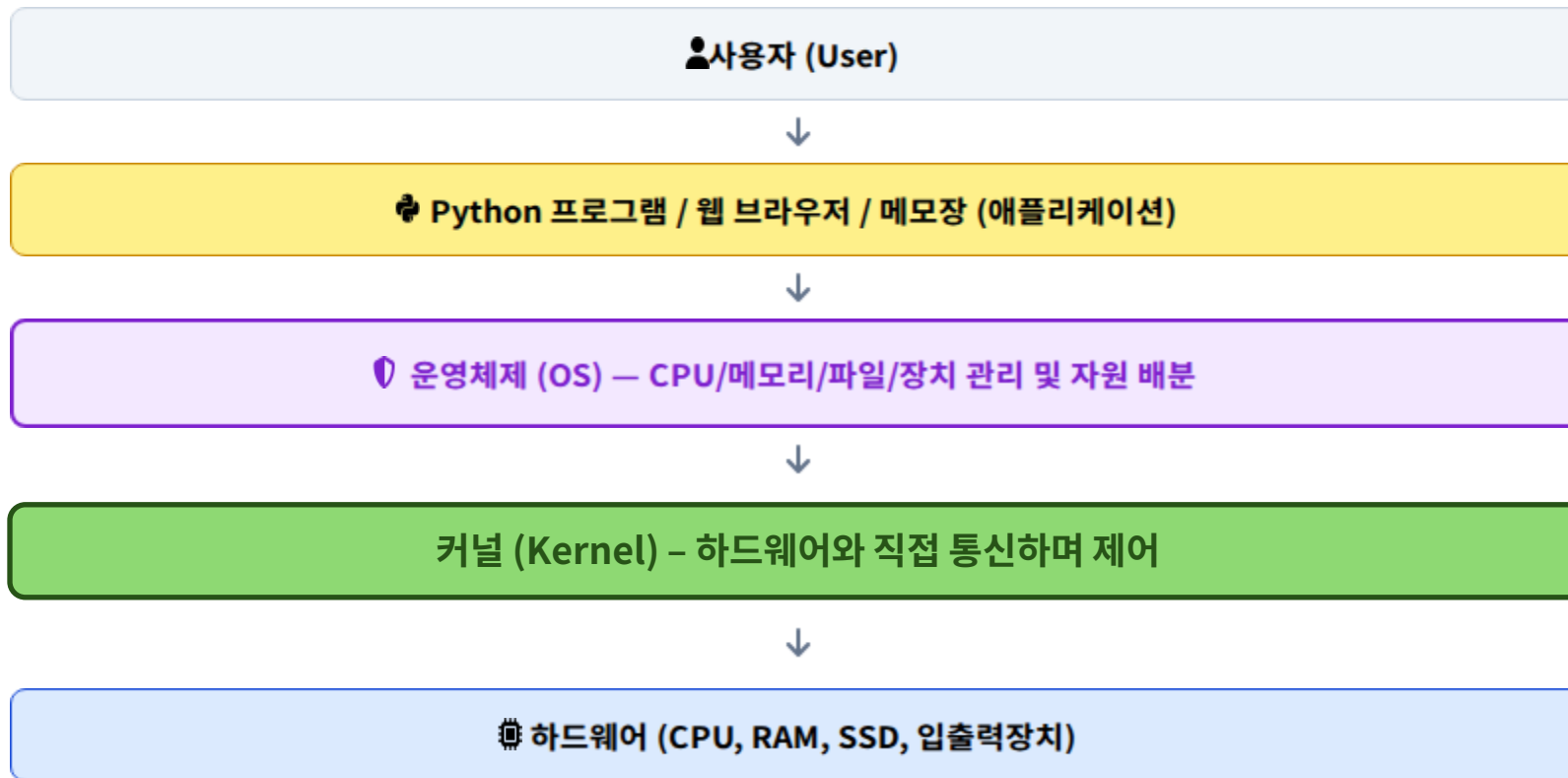


ALU는 레지스터의
값으로 계산

- CPU 칩 안에 들어있는 초고속 임시 저장소
- 연산에 필요한 데이터와 주소 저장
- CPU가 명령 실행 및 연산 수행을 위해 사용

01 Review

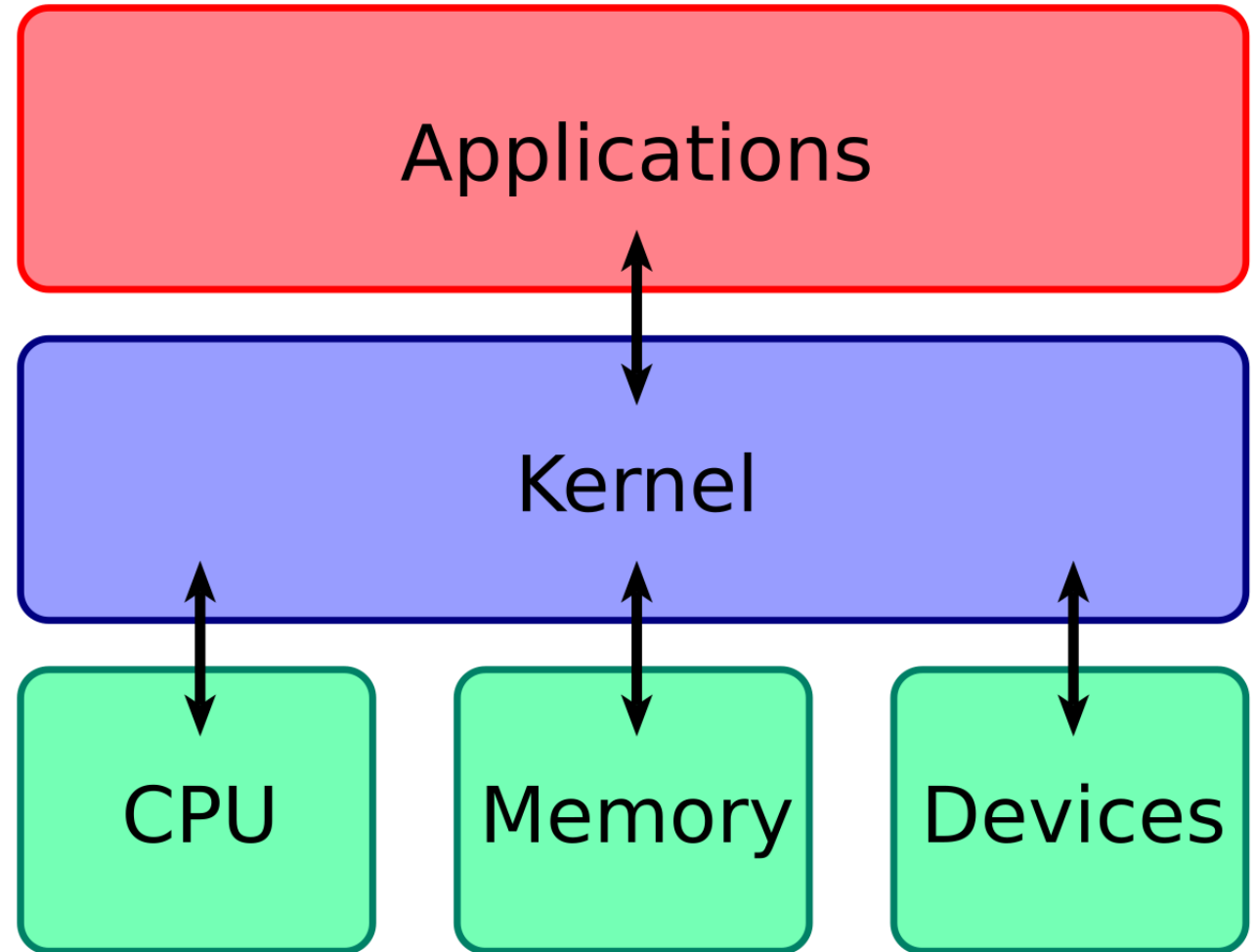
시스템 계층 구조 System Hierarchy



01 Review

커널 Kernel

- 운영체제의 핵심인 소프트웨어
- 하드웨어 자원을 자원이 필요한 프로그램에 나눠줌
- 작업 제어, 메모리 제어, 시스템 콜 등 수행
- 하드웨어에 직접적으로 명령
- OS마다 커널 구조가 다 다름



01 Review

Python이 내부에서 실행되는 과정



CPython 표준 기준

우리가 가장 많이 쓰는 표준 Python(CPython)은 소스코드를 .pyc 형태의 바이트코드로 먼저 바꾼 뒤, 파이썬 가상 머신(PVM)을 통해 CPU가 이해할 수 있는 명령어로 해석해 나갑니다.

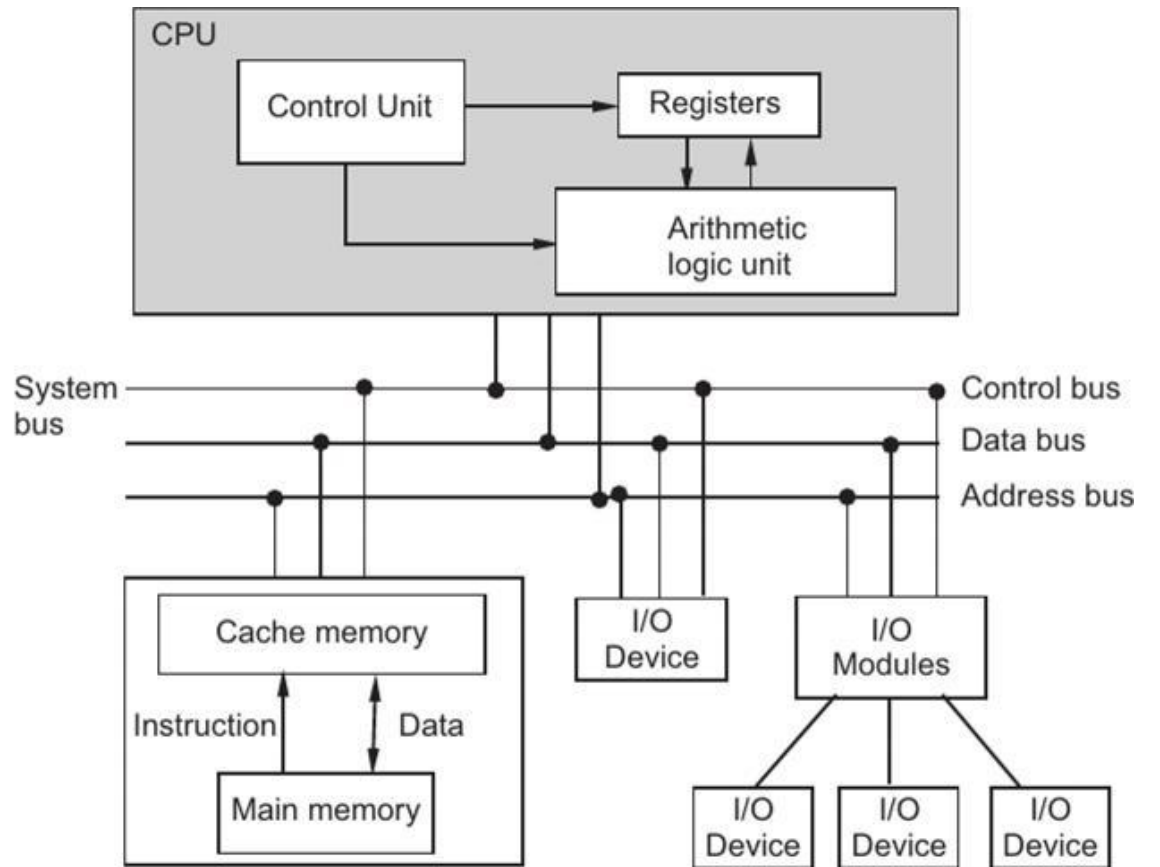
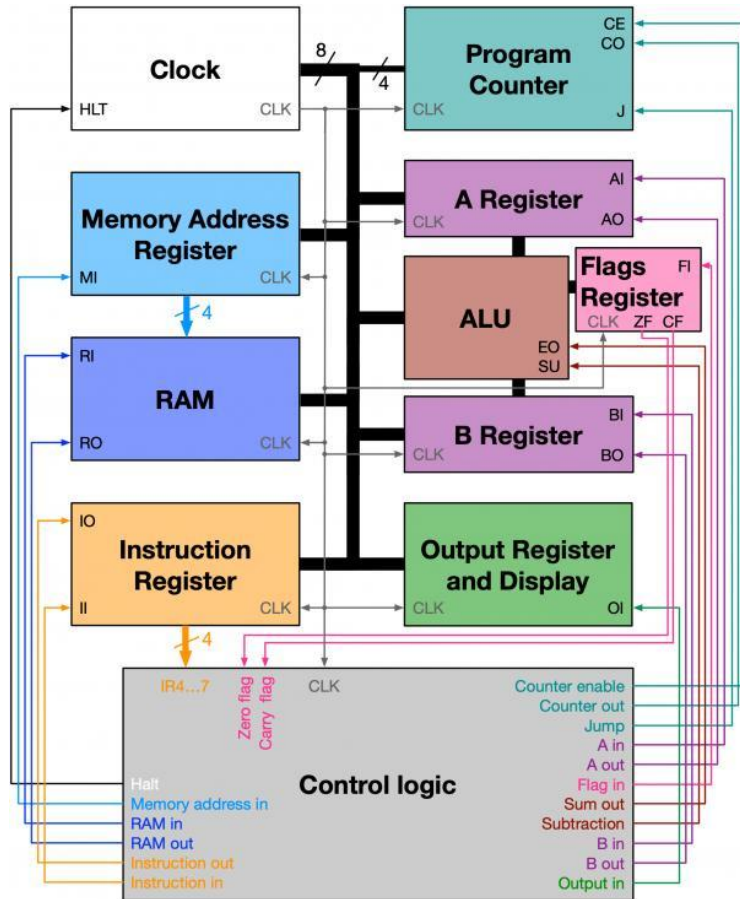
CPU가 파이썬 소스 코드를 직접 읽는 것은 절대 아닙니다!

02

CPU 심화

Central Processing Unit (Advanced)

오늘 배울거



오늘은 이런거 배울거임
걱정마셈 차근차근 알아보면 안어려움

CPU의 내부 핵심 부품

[CPU Block Diagram]

Control Unit (제어장치)



ALU (산술·논리 연산장치)



Registers (레지스터)



Cache (캐시 메모리)

Control Unit (제어장치)

메모리에서 명령어를 읽어들이고, 무엇을 해야 할지 해석하여 다른 부품들에게 신호를 보냄

ALU (산술논리연산장치)

더하기, 빼기 같은 산술 연산과 참/거짓을 따지는 논리 연산을 실제로 수행하는 계산기

Registers & Cache

CPU 안에서 연산에 필요한 데이터나 명령어를 빠른 속도로 기억하는 임시 공간

클럭 (Clock)

- 컴퓨터 내부의 부품들이 1초에 몇 번씩 움직일지에 대한 신호
- Hz(1초당 신호 수)로 측정
- 클럭 신호에서 꼭 연산을 할 필요는 없음
3GHz 클럭은 1초에 30억번의 연산 기회가 주어진다는 뜻

클럭 (Clock)의 개념

컴퓨터 내부의 모든 부품이 일사불란하게 움직이도록 박자를 맞춰주는 **초고속 메트로놈** 또는 **심장박동**입니다.

Hz(헤르츠) 단위로 측정되며, 3.5 GHz라면 1초에 35억 번의 박동(틱)이 발생한다는 뜻입니다.

⚠주의: 클럭 속도(GHz)가 높다고 무조건 실제 체감 속도가 빠른 것은 아닙니다! 한 번에 처리하는 일의 양(아키텍처 효율)이 더 중요합니다.

캐시랑 레지스터 왜 쓰나요 (1)

- 1970년대의 CPU는 캐시가 없었음
(레지스터만 있었음)
- CPU도 느리고, 메모리도 작았음
- CPU와 RAM의 속도가 비슷해서 큰 문제가 없었다
- 그러다가 기술의 발전으로 CPU의 클럭이 급격히 상승했고, 이 속도를 RAM이 따라오지 못하며 CPU는 RAM에서 데이터를 불러올 때까지 대부분 시간을 대기하게 됨

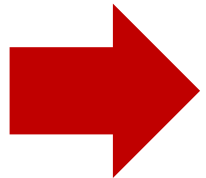
캐시가 생기기 전의 CPU 동작



캐시랑 레지스터 왜 쓰나요 (2)

첫 번째 해결방법 <레지스터 늘리기>

- 레지스터를 8개, 16개, 심지어 64개까지 늘려봄
- 근데 레지스터는 CPU가 이름을 직접 불러줘야됨
- 레지스터를 100만개 만든다면 명령어가 쓸데없이 길고 복잡해질 수 있음
- 명령 해석 과정도 어려워지고, 소비전력도 폭증하게 됨



이건 좀 아닌듯



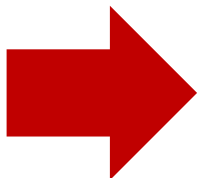
캐시랑 레지스터 왜 쓰나요 (3)

두 번째 해결방법 <캐시를 추가하자>

- 핵심 아이디어 : CPU가 자주 쓰는 데이터를 자동으로 복사해놓는 공간을 따로 만들자
- CPU는 적은 수의 레지스터만 알면 되며, 실행 속도도 향상됨
- 요리사는 매번 냉장고를 열지 않고, 사용할 재료를 조리대에 미리 꺼내 두는 것과 비슷
그래야 작업이 빨라지고 효율이 올라감



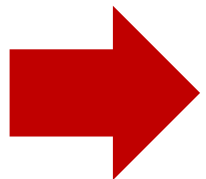
- CPU: 요리사
- Register: 요리사의 손
- Cache: 조리대에 미리 꺼내놓은 재료
- RAM: 재료 저장해놓은 냉장고
- HDD/SSD: 집 밖의 식료품가게



레지스터는 CPU의 작업공간, 캐시는 RAM의 복사본

L1 L2 L3 캐시는 왜 쓰나요

- 캐시를 추가해도 CPU는 계속 빨라졌음
- CPU가 빨라질수록 더 큰 저장공간 필요 (처리 속도가 증가하므로)
- 그런데 큰 저장공간은 내부의 복잡도가 올라가므로 속도가 상대적으로 느려짐
- 그래서 캐시를 크기에 따라 3개로 나눔



빠르고 큰 저장공간 탄생

Locality

- 프로그램은 잘 생각해보면 대체로 특정한 데이터와 그 주변 데이터를 반복해서 사용하는 경향이 있음
- 시간 지역성과 공간 지역성을 이용해서 반복해서 사용되는 데이터를 캐시에 저장함
- 캐시가 없으면 매번 냉장고에 다녀와야 함 (실행속도 느려짐)

1. 시간 지역성 (Temporal Locality)

"방금 사용한 데이터는 잠시 후 또 사용할 확률이 높다."

예시: 반복문 안에서 계속 호출되는 변수나 카운터 등은 캐시에 남겨두는 것이 유리합니다.

2. 공간 지역성 (Spatial Locality)

"특정 데이터 근처에 있는 데이터도 곧 함께 사용될 확률이 높다."

예시: 배열(Array)의 첫 번째 원소를 읽었다면, 그 옆에 있는 두 번째, 세 번째 원소도 차례대로 읽을 가능성이 매우 큼니다.

CPU 내부 레지스터

Program Counter (PC)

'프로그램 카운터'

다음에 실행해야 할 **명령어의 메모리 주소**를 기억하고 있는 가장 중요한 레지스터.
매 명령어가 실행될 때마다 자동으로 1씩 증가

Instruction Register (IR)

'명령어 레지스터'

방금 메모리에서 읽어들이는 **실제 명령어**를 잠시 임시로 보관하여
제어장치가 해석할 수 있도록 대기하는 공간

General Purpose Register

'범용 레지스터'

데이터나 계산 중간 결과, 주소 등을 자유롭게 담아둘 수 있는
다목적 저장 공간

Stack Pointer (SP) 등

'특수 목적 레지스터'

함수 호출이나 임시 메모리 스택 구조를 추적하는 데 사용되는
주소 관리 레지스터 (특수 목적이라고 생각하면 됨)

System BUS

시스템 버스의 종류

CPU, 메모리, 입출력장치 등 장치 사이사이 데이터가 이동하는 물리적인 통로 (여기선 CPU ↔ RAM 만 다름)

Address Bus (주소 버스)

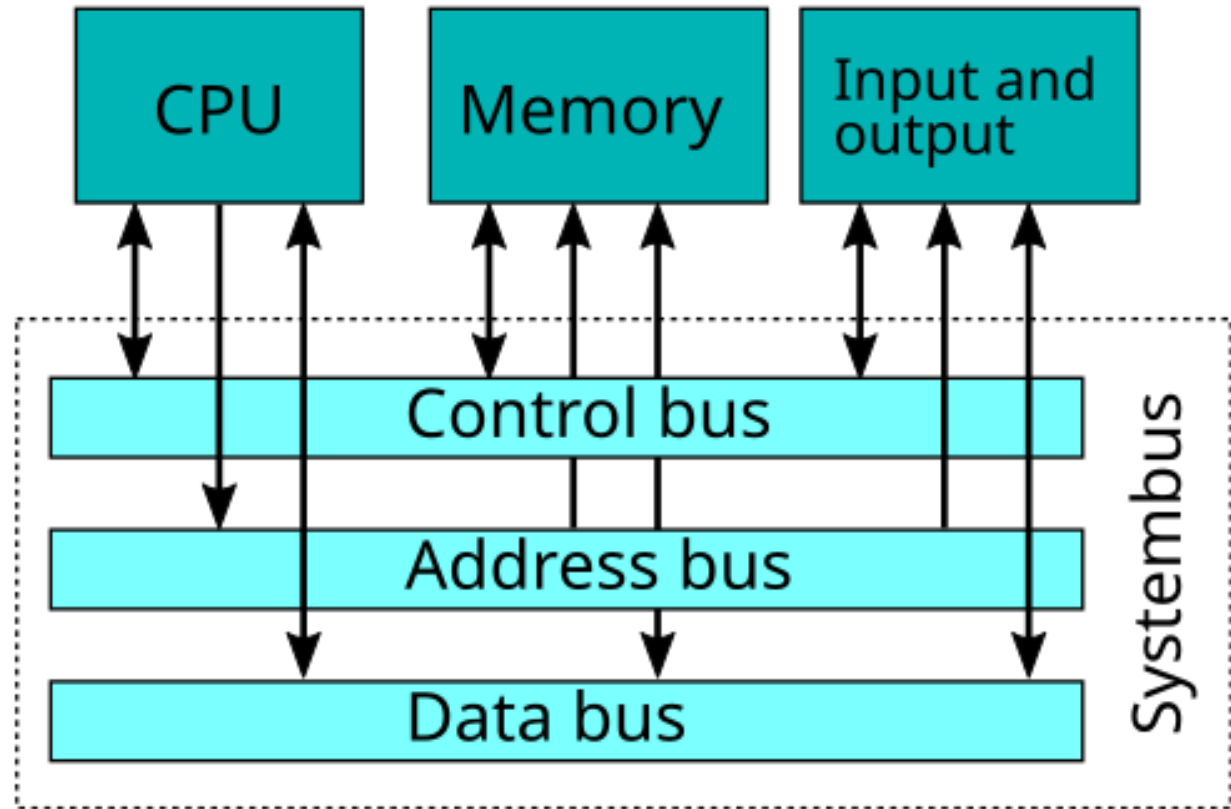
CPU가 지금 접근하려는 메모리의 주소 전달 (단방향, CPU → RAM)

Data Bus (데이터 버스)

진짜 데이터가 오고가는 통로 (양방향)

Address Bus (주소 버스)

읽기/쓰기 등 제어 신호와 지시 신호 전달 (양방향)



03

CPU 동작

Instruction Cycle

명령어

Instructions

🔗 명령어의 구조

CPU가 수행하는 모든 명령은 아주 단순한 형태의 비트 조합으로 이루어져 있음

[Opcode (연산 코드)] + [Operand (오퍼랜드/
데이터)]

- **Opcode:** 무슨 연산을 할까? (더해라, 읽어라 등)
- **Operand:** 누구를 대상으로 할까? (데이터나 주소)

📖 주요 기초 명령어 예시

- **LOAD:** 메모리에서 레지스터로 데이터를 가져옴
- **STORE:** 레지스터의 결과를 메모리에 저장함
- **ADD / SUB:** 덧셈, 뺄셈 연산 수행
- **JUMP:** 다른 명령어나 조건으로 순서를 바꿈

RISC & CISC (참고)

Reduced Instruction Set Computer (RISC)

'명령어의 길이가 모두 같음'

해석이 쉽고, 제어장치가 간단함 (길이가 다 같으므로)
Apple M1~M4가 대표적

Complex Instruction Set Computer (CISC)

'명령어별로 길이가 모두 다름'

해석이 비교적 어렵고, 제어장치가 복잡함
Intel core, Intel Xeon, AMD RYZEN이 대표적

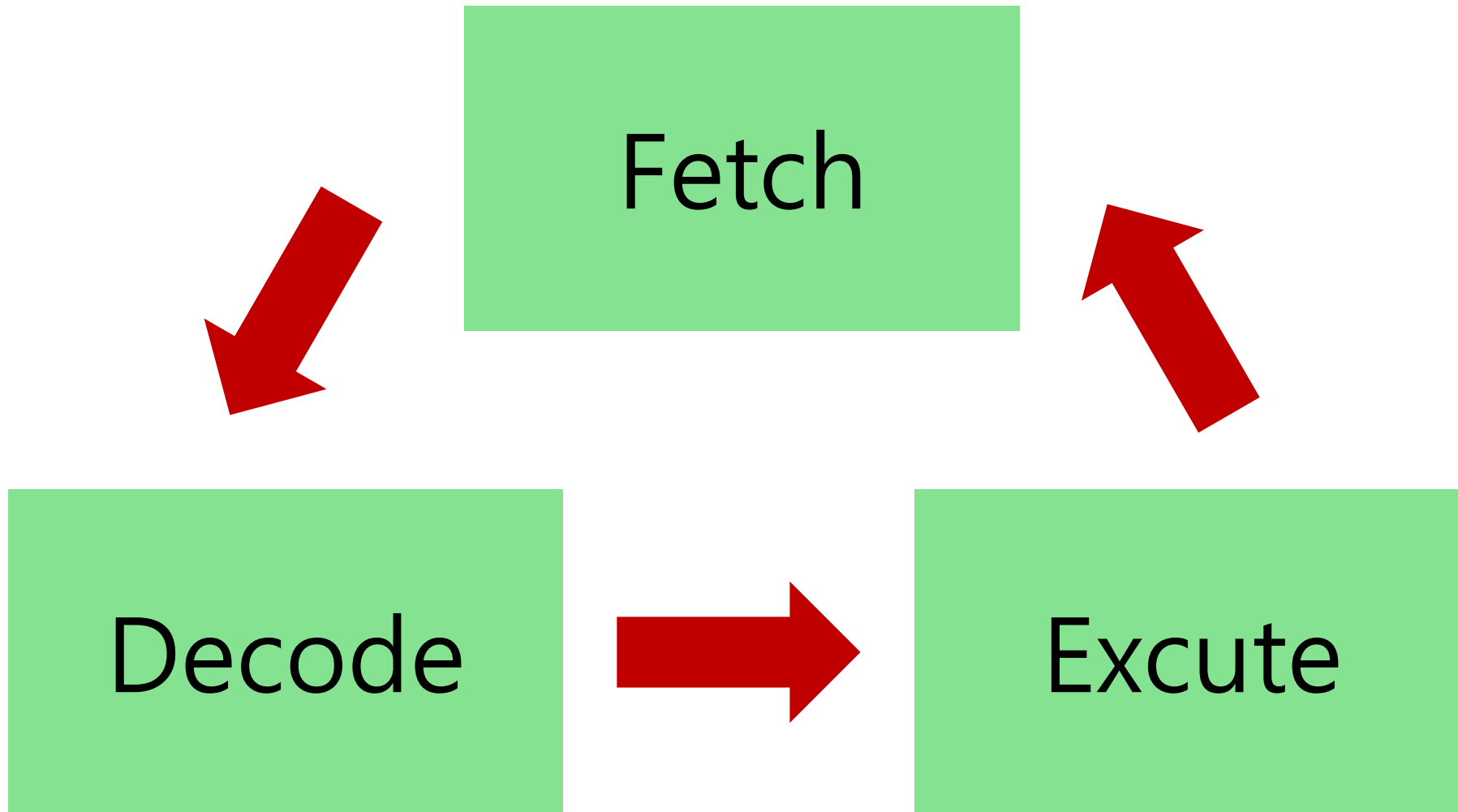
CISC

non-fixed size
of instruction

RISC

fixed size
of instruction

F-D-E Cycle



F: Fetch, 인출

→ Fetch 단계에서 일어나는 일

1. 프로그램 카운터(PC)가 가리키는 메모리 주소를 주소 버스에 실어 보냄
2. 제어장치가 메모리에 '읽기' 신호를 제어 버스를 통해 보냄
3. RAM에 있던 명령어가 데이터 버스를 타고 넘어와 CPU 내부의 명령어 레지스터(IR)에 저장됨

[CPU] PC = 0x1000

▼ (주소 전송)

[RAM] 0x1000 번지 명령어 확인

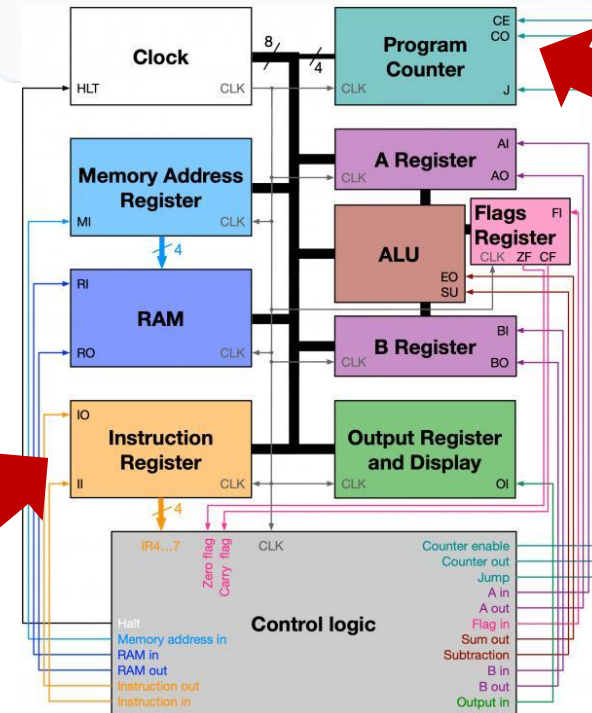
▼ (데이터 전송)

[CPU] IR 레지스터에 명령어 적재완료!

이후 PC 값은 다음 명령어를 가리키도록 자동으로 증가합니다. (PC = PC + 4 등)

명령어 레지스터

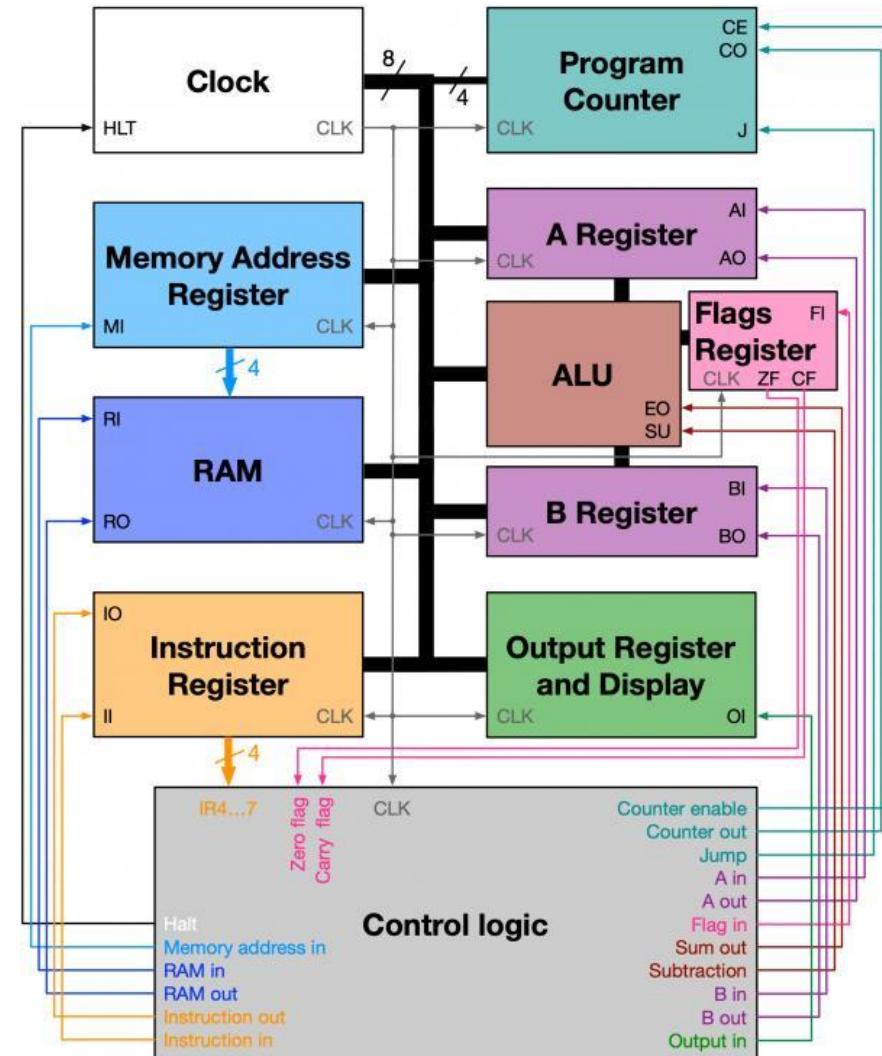
프로그램 카운터



D: Decode, 해독

QDecode 단계에서 일어나는 일

1. 명령어 레지스터(IR)에 담긴 0과 1의 비트 조합(명령어)을 제어장치(Control Unit)가 넘겨받음
2. 제어장치가 이 비트를 쪼개어 "이것은 덧셈(ADD) 명령어이며, 대상은 레지스터 A와 B이다"라고 해석
3. 연산에 필요한 추가 데이터가 메모리에 있다면 어디서 가져와야 할지 주소를 파악



E: Excute, 실행

Execute 단계에서 일어나는 일

1. 제어장치가 해독된 내용에 맞추어 적절한 제어 신호를 보냄
2. 산술논리연산장치(ALU)가 실제 계산을 수행하거나,
레지스터 간 이동, 혹은 메모리 저장(STORE)을 실행
3. 연산 결과는 레지스터에 기록되거나 메모리에 반영

Python과 Instruction 연결 (1)

- 파이썬 코드를 실행하면 인터프리터가 바이트코드로 변환하고, PVM이 이를 해석하여 CPU에서 실행
- Fetch 단계: RAM에서 기계어로 된 명령어 가져옴
- Decode 단계: 불러온 명령어를 해석해서 숫자를 더한 뒤 출력하라는 명령이라는걸 분석함
- Excute 단계: 1번 레지스터에 10, 2번 레지스터에 20을 불러온 후 ALU를 통해 두 레지스터를 더한 값을 3번 레지스터에 저장
이후 3번 레지스터의 값을 출력장치로 보냄

Python 코드 예시

a = 10

b = 20

print(a + b)

개념적 CPU 실행 대응

LOAD R1, 10 # 10을 가져옴

LOAD R2, 20 # 20을 가져옴

ADD R3, R1, R2 # 더해서 R3에 저장

STORE R3, [출력장치]

Python과 Instruction 연결 (2)

Python 코드 예시

a = 10

b = 20

print(a + b)

개념적 CPU 실행 대응

LOAD R1, 10 # 10을 가져옴

LOAD R2, 20 # 20을 가져옴

ADD R3, R1, R2 # 더해서 R3에 저장

STORE R3, [출력장치]

1번째 사이클

Fetch: RAM에서 “LOAD R1, 10” 인출

Decode: 이 명령어는 1번 레지스터에 10이라는 값을 넣으라는 뜻임

Excute: 1번 레지스터에 10이라는 값을 넣음

Program Counter 1 증가

2번째 사이클

Fetch: RAM에서 “LOAD R2, 20” 인출

Decode: 이 명령어는 2번 레지스터에 20이라는 값을 넣으라는 뜻임

Excute: 2번 레지스터에 20이라는 값을 넣음

Program Counter 1 증가

3번째 사이클

Fetch: RAM에서 “ADD R3, R1, R2” 인출

Decode: 이 명령어는 3번 레지스터에 1번과 2번 레지스터의 값을 더해서 넣으라는 뜻임

Excute: 1번 레지스터의 값과 2번 레지스터의 값을 더해서 3번 레지스터에 넣음

Program Counter 1 증가

4번째 사이클

Fetch: RAM에서 “STORE R3, [출력장치]” 인출

Decode: 이 명령어는 3번 레지스터의 값을 출력장치에 저장(즉, 출력)하라는 뜻임

Excute: 3번 레지스터의 값을 출력장치에 저장

Program Counter 1 증가